$$
\begin{array}{rl}
e ::= x \mid c \mid \text{true} \mid \text{false} \mid k_{\#n} \mid () & \text{constants} \\
\mid \textbf{let}\ x = e_1\ \textbf{in}\ e_2 & \text{let binding} \\
\mid \textbf{uniform}(e_1, e_2) \mid \textbf{gaussian}(e_1, e_2) \mid \textbf{exponential}(e) \mid \ldots & \text{continuous distributions} \\
\mid \textbf{discrete}(p_0, \ldots, p_{n-1}) & \text{discrete distribution} \\
\mid e_1 < e_2 \mid e_1 \leq e_2 \mid e_1 <_{\#n} e_2 \mid e_1 \leq_{\#n} e_2 & \text{comparisons} \\
\mid e_1 \,\&\&\, e_2 \mid e_1 \,||\, e_2 \mid \text{not } e & \text{Boolean operations} \\
\mid \textbf{if}\ e_1\ \textbf{then}\ e_2\ \textbf{else}\ e_3 & \text{conditional} \\
\mid (e_1, e_2) \mid \textbf{fst}\ e \mid \textbf{snd}\ e & \text{pairs} \\
\mid \textbf{fun}\ x \ \to\ e \mid e_1\ e_2 \mid \textbf{fix}\ f\ x := e & \text{functions} \\
\mid \textbf{observe}\ e \mid e_1; e_2 & \text{effects} \\
\mid \text{nil} \mid e_1 :: e_2 \mid (\textbf{match}\ e\ \textbf{with} \mid \text{nil} \to e_{\text{nil}} \mid e_1 :: e_2 \to e_{\text{cons}}\ \textbf{end}) & \text{lists} \\
\mid \textbf{diverge} & \text{diverge}
\end{array}
$$

Fig. 5. Syntax of the Slice language. Slice supports a variety of continuous distributions with one and two parameters; for a full list of supported distributions, see Appendix C.

3.1 The Slice Language

The syntax in Figure 5 extends a standard higher-order functional core with probabilistic choices. Beyond constants, variables, comparisons, Boolean operators, conditionals, pairs, and lists, Slice offers first-class functions, recursion, and observations. The construct $\textbf{observe}(e)$ conditions on e holding. Finite-domain constants $k_{\#n}$ range over $\{0_{\#n}, \ldots, (n-1)_{\#n}\}$ as discussed below. For random sampling, we include the usual discrete distributions and any continuous distribution with a tractable cumulative distribution function (one- or two-argument variants such as exponential, laplace, uniform, gaussian, beta, gamma). Adding further distributions is straightforward as long as their CDFs are known; §C lists the ones implemented.

The expression $\textbf{discrete}(p_0, \ldots, p_{n-1})$ requires a nonempty probability vector: $n \geq 1$, every $p_i \geq 0$, and $\sum_i p_i = 1$. It returns $k_{\#n}$ with probability p_k.

These are ideal real-valued premises. The implementation separately validates finite binary64 weights. It normalizes compiler-generated weights only within a tight allowance for accumulated CDF arithmetic, and parses source vectors within a documented 10^{-4} rounded-input allowance. These engineering policies do not replace the exact premise above: binary64 evaluation and normalization do not by themselves establish exact preservation of ideal real-valued probabilities, which requires a separate numerical error analysis.

3.2 Types

We formalize the types that Slice supports. A *value* set V is a finite set of reals or $\top$. Elements of $\{<x \mid x \in \mathbb{R}\} \cup \{\leq x \mid x \in \mathbb{R}\}$ are called *cuts*. Cut sets induce intervals partitioning the reals as follows: Given a non-empty cut set $B = \{\sim_1 x_1, \ldots, \sim_n x_n\}$ with $x_1 \leq \ldots \leq x_n$, we define the set $\text{Intervals}(B)$ of intervals it induces as expected, e.g., $\text{Intervals}(\{<0.1, \leq 0.5\}) = \{(-\infty, 0.1), [0.1, 0.5], (0.5, \infty)\}$. The strict and non-strict cuts differ only in which side contains the boundary point: $\text{Intervals}(\{<c\}) = \{(-\infty, c), [c, \infty)\}$, whereas $\text{Intervals}(\{\leq c\}) = \{(-\infty, c], (c, \infty)\}$. Thus a boundary generated by $e < c$ places c on the right, while a boundary generated by $e \leq c$ places c on the left. Moreover, we let $\text{Intervals}(\emptyset) = \{(-\infty, \infty)\}$ and $\text{Intervals}(\top) = 2^{\mathbb{R}}$. We denote by $\text{Intervals}(B)_i$ the i-th interval of $\text{Intervals}(B)$, assuming $\text{Intervals}(B)$ is 0-indexed, i.e., $i \in \{0, ..., n\}$. We denote the set of all cut- and value sets, respectively, by $\mathcal{B}$ and $\mathcal{V}$.

Intuitively, $\text{recovers}(B, V)$ means that the discretization induced by B is lossless on V: from the resulting interval index, the original value in V can be uniquely recovered. For example, $\textbf{exponential}(e)$ can be discretized only when e takes finitely many recoverable values. Thus V must be finite and B must separate those values so the discretized program can determine which one occurred. §5 revisits this invariant. The rule for two-argument distributions is analogous.

Comparisons. For $e < c$ with $e : \textbf{float}[B_1; \top]$ and $c : \textbf{float}[B_2; \{c\}]$, the discretized e must still decide the predicate, so B_1 must include $<c$. If c can take any value in V_2, every $<v$ for $v \in V_2$ must appear. Symmetrically, for $c < e$ with $e : \textbf{float}[B_2; V_2]$, B_2 must contain $\leq c$ so the discretized intervals know on which side of c they fall. For example, $2.3 < e$ requires cuts separating $(-\infty, 2.3]$ from $(2.3, \infty)$; finer partitions are allowed but not required.

In general, the typing rule for $e_1 < e_2$ requires that the expressions share a cut set B, and that their respective value sets V_1, V_2 are related in a way that allows the comparison to be determined after discretization, via the $\text{answers}_<(B, V_1, V_2)$ relation:

Definition 3.3. Let $B \in \mathcal{B}$ and $V_1, V_2 \in \mathcal{V}$. We say that $\text{answers}_<(B, V_1, V_2)$ if (1) $B = \top$, or (2) $B \neq \top$ and $V_2 \neq \top$ and $\forall x \in V_2,\ {<x} \in B$, or (3) $B \neq \top$ and $V_1 \neq \top$ and $\forall x \in V_1,\ {\leq x} \in B$.

To understand this definition, we first want to introduce two different perspectives on the B set. The first perspective is that B specifies a set of intervals, such that values of the expression will be mapped to their respective interval index. The second perspective is that this discretized index *still contains enough information to evaluate all comparisons in B when viewed as predicates.* That is, if B contains $<x$ then the discretized value of $e : \textbf{float}[B; V]$ still contains enough information to answer $e < x$ precisely: if after discretization, e has an interval index below the interval containing x, then the comparison is true, and otherwise it is false.

Now, if $e_1 : \textbf{float}[B; V_1]$ and $e_2 : \textbf{float}[B; V_2]$, then $\text{answers}_<(B, V_1, V_2)$ ensures that the comparison result can still be determined after discretization because:

(1) If $B = \top$, then no discretization is performed, and the comparison can just be performed on the real-valued expression.
(2) If $B \neq \top$ and $V_2 \neq \top$, then the expression is discretized into intervals as determined by B, but in this case the right-hand side can only take on values in V_2. In this case, all $<x$ for $x \in V_2$ must be contained in B, which means that the value of e_2 can be exactly recovered from its interval index. Then, given the previous observation, we can determine the comparison result by comparing the interval index of e_1 to the interval index of e_2.
(3) If $B \neq \top$ and $V_1 \neq \top$, then the situation is similar to the previous case.

Note that $B = \top$ iff $V = \top$ accounts for comparisons where a continuously ranging expression is compared against arbitrary real values not statically known, such as other continuous distributions, in which case no discretization can be performed. Additionally, we insist that $B_1 = B_2$, which ensures that both comparands will be discretized to the same type, or not discretized at all.

Finally, for non-strict comparisons, the rule is analogous to Definition 3.3:

Definition 3.4. Let $B \in \mathcal{B}$ and $V_1, V_2 \in \mathcal{V}$. We say that $\text{answers}_\leq(B, V_1, V_2)$ if (1) $B = \top$, or (2) $B \neq \top$ and $V_2 \neq \top$ and $\forall x \in V_2,\ {\leq x} \in B$, or (3) $B \neq \top$ and $V_1 \neq \top$ and $\forall x \in V_1,\ {<x} \in B$.

Subtyping. The type system includes a subtyping relation that captures when one type can be replaced by another. The subtyping rules are shown in Figure 8. For float types $\textbf{float}[B; V]$, subtyping follows the lattice ordering on V: an expression known to range over some set of values can also be considered to range over a superset of those values. For B, we require equality to make sure that the discretized representation of the expression is not changed by subtyping.

All other typing rules are standard and depicted in Figure 7 and §D, respectively.

$$\textsc{Var}\ \frac{}{\Gamma, x : \tau \vdash x : \tau} \qquad \textsc{Let}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \mathbf{let}\ x = e_1\ \mathbf{in}\ e_2 : \tau_2} \qquad \textsc{If}\ \frac{\Gamma \vdash e_1 : \mathbf{bool} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \mathbf{if}\ e_1\ \mathbf{then}\ e_2\ \mathbf{else}\ e_3 : \tau}$$

$$\textsc{Observe}\ \frac{\Gamma \vdash e : \mathbf{bool}}{\Gamma \vdash \mathbf{observe}\ e : \mathbf{unit}} \qquad \textsc{Discrete}\ \frac{n \geq 1 \qquad \forall i \in \{0, \ldots, n-1\}.\ p_i \geq 0 \qquad \sum_{i=0}^{n-1} p_i = 1}{\Gamma \vdash \mathbf{discrete}(p_0, \ldots, p_{n-1}) : \mathbf{fin}(n)}$$

$$\textsc{FinConst}\ \frac{0 \leq k < n}{\Gamma \vdash k_{\#n} : \mathbf{fin}(n)} \qquad \textsc{FinLess}\ \frac{\Gamma \vdash e_1 : \mathbf{fin}(n) \qquad \Gamma \vdash e_2 : \mathbf{fin}(n)}{\Gamma \vdash e_1 <_{\#n} e_2 : \mathbf{bool}} \qquad \textsc{FinLessEq}\ \frac{\Gamma \vdash e_1 : \mathbf{fin}(n) \qquad \Gamma \vdash e_2 : \mathbf{fin}(n)}{\Gamma \vdash e_1 \leq_{\#n} e_2 : \mathbf{bool}}$$

$$\textsc{Diverge}\ \frac{}{\Gamma \vdash \mathbf{diverge} : \tau}$$

Fig. 7. Typing rules for general language constructs

$$\textsc{Sub-Refl}\ \frac{\tau \in \{\mathbf{int}, \mathbf{bool}, \mathbf{fin}(n)\}}{\tau <: \tau} \qquad \textsc{Sub-Float}\ \frac{B_1 = B_2 \qquad V_1 \sqsubseteq V_2}{\mathbf{float}[B_1; V_1] <: \mathbf{float}[B_2; V_2]} \qquad \textsc{Sub-Pair}\ \frac{\tau_1 <: \tau_1' \qquad \tau_2 <: \tau_2'}{\tau_1 * \tau_2 <: \tau_1' * \tau_2'}$$

$$\textsc{Sub-List}\ \frac{\tau <: \tau'}{\mathbf{list}(\tau) <: \mathbf{list}(\tau')} \qquad \textsc{Sub-Arrow}\ \frac{\tau_1' <: \tau_1 \qquad \tau_2 <: \tau_2'}{\tau_1 \to \tau_2 <: \tau_1' \to \tau_2'} \qquad \textsc{Sub-Type}\ \frac{\tau_1 <: \tau_2 \qquad \Gamma \vdash e : \tau_1}{\Gamma \vdash e : \tau_2}$$

Fig. 8. Subtyping relation. For both cut- and value sets, we let $d \sqsubseteq d'$ iff $\begin{cases} d \subseteq d' & \text{if } d \neq \top \text{ and } d' \neq \top \\ d' = \top & \text{otherwise .} \end{cases}$

4 Type Inference

We now describe our type inference algorithm, whose goal is to annotate each subexpression of a program with its most precise type. Specifically, we focus on expressions of type $\mathbf{float}[B; V]$ and show how Slice populates the cut set B and value set V, which later determine how and whether the expression can be discretized. Type inference for all other types is standard and follows the classic Hindley-Milner algorithm. For float-typed expressions, we use a two-phase approach: the first phase traverses the program once to generate constraints over cut and value sets; the second phase solves those constraints to produce a fully annotated program.

The distinction between typing and type inference is important. The correctness argument cleanly separates into two pieces: (1) type inference produces a typing derivation matching our typing rules, and (2) given such a typing derivation, discretization is semantics preserving. Thus the proof of discretization does not have to reason directly about the type inference procedure.

4.1 Constraint Generation

We use the constraint language in Figure 9 to express requirements over the type structure and float refinements. We adopt the framework of [38], introducing a new judgment of the form $\Gamma \vdash e : \tau \mid C$, which reads as *in the environment* Γ, *e has type* τ *and produces the set of constraints* C. In this judgment, Γ and e are inputs to the type inference algorithm, while τ and C are outputs: the algorithm infers the type τ for expression e and generates the constraints C that must hold for this typing to be valid. The inference rules formalize which constraints must be generated and later solved for a given expression e to have inferred type τ. Figure 10 gives the constraint generation rules for Slice. We focus on a small but representative subset of constructs in Slice, capturing the key ideas behind constraint generation for the *full* language.

$$\textsc{Var}\ \frac{}{\Gamma, x : \tau \vdash x : \tau} \qquad \textsc{Let}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \mathbf{let}\ x = e_1\ \mathbf{in}\ e_2 : \tau_2} \qquad \textsc{If}\ \frac{\Gamma \vdash e_1 : \mathbf{bool} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \mathbf{if}\ e_1\ \mathbf{then}\ e_2\ \mathbf{else}\ e_3 : \tau}$$

$$\textsc{Discrete}\ \frac{n \geq 1 \qquad \forall i \in \{0, \ldots, n-1\}.\ p_i \geq 0 \qquad \sum_{i=0}^{n-1} p_i = 1}{\Gamma \vdash \mathbf{discrete}(p_0, \ldots, p_{n-1}) : \mathbf{fin}(n)} \qquad \textsc{LessEq}\ \frac{\Gamma \vdash e : \mathbf{int}}{\Gamma \vdash e \leq i : \mathbf{bool}}$$

$$\textsc{Pair}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash (e_1, e_2) : \tau_1 * \tau_2} \qquad \textsc{Fst}\ \frac{\Gamma \vdash e : \tau_1 * \tau_2}{\Gamma \vdash \mathbf{fst}\ e : \tau_1} \qquad \textsc{Snd}\ \frac{\Gamma \vdash e : \tau_1 * \tau_2}{\Gamma \vdash \mathbf{snd}\ e : \tau_2} \qquad \textsc{Fun}\ \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \mathbf{fun}\ x\ \rightarrow\ e : \tau_1 \rightarrow \tau_2}$$

$$\textsc{App}\ \frac{\Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ e_2 : \tau_2} \qquad \textsc{True}\ \frac{}{\Gamma \vdash \text{true} : \mathbf{bool}} \qquad \textsc{False}\ \frac{}{\Gamma \vdash \text{false} : \mathbf{bool}}$$

$$\textsc{And}\ \frac{\Gamma \vdash e_1 : \mathbf{bool} \qquad \Gamma \vdash e_2 : \mathbf{bool}}{\Gamma \vdash e_1\ \&\&\ e_2 : \mathbf{bool}} \qquad \textsc{Or}\ \frac{\Gamma \vdash e_1 : \mathbf{bool} \qquad \Gamma \vdash e_2 : \mathbf{bool}}{\Gamma \vdash e_1\ ||\ e_2 : \mathbf{bool}} \qquad \textsc{Not}\ \frac{\Gamma \vdash e : \mathbf{bool}}{\Gamma \vdash \text{not}\ e : \mathbf{bool}}$$

$$\textsc{Unit}\ \frac{}{\Gamma \vdash () : \mathbf{unit}} \qquad \textsc{FinConst}\ \frac{0 \leq k < n}{\Gamma \vdash k_{\#n} : \mathbf{fin}(n)} \qquad \textsc{FinLess}\ \frac{\Gamma \vdash e_1 : \mathbf{fin}(n) \qquad \Gamma \vdash e_2 : \mathbf{fin}(n)}{\Gamma \vdash e_1 <_{\#n} e_2 : \mathbf{bool}}$$

$$\textsc{FinLessEq}\ \frac{\Gamma \vdash e_1 : \mathbf{fin}(n) \qquad \Gamma \vdash e_2 : \mathbf{fin}(n)}{\Gamma \vdash e_1 \leq_{\#n} e_2 : \mathbf{bool}} \qquad \textsc{Observe}\ \frac{\Gamma \vdash e : \mathbf{bool}}{\Gamma \vdash \mathbf{observe}\ e : \mathbf{unit}} \qquad \textsc{Seq}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash e_1; e_2 : \tau_2}$$

$$\textsc{Fix}\ \frac{\Gamma, f : \tau_1 \rightarrow \tau_2, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \text{fix}\ f\ x := e : \tau_1 \rightarrow \tau_2} \qquad \textsc{Nil}\ \frac{}{\Gamma \vdash \text{nil} : \mathbf{list}(\tau)} \qquad \textsc{Cons}\ \frac{\Gamma \vdash e_1 : \tau \qquad \Gamma \vdash e_2 : \mathbf{list}(\tau)}{\Gamma \vdash e_1 :: e_2 : \mathbf{list}(\tau)}$$

$$\textsc{Match}\ \frac{\Gamma \vdash e : \mathbf{list}(\tau_1) \qquad \Gamma \vdash e_1 : \tau_2 \qquad \Gamma, h : \tau_1, t : \mathbf{list}(\tau_1) \vdash e_2 : \tau_2}{\Gamma \vdash \text{match}\ e\ \text{with nil} \rightarrow e_1 \mid h :: t \rightarrow e_2\ \text{end} : \tau_2}$$

$$\textsc{Diverge}\ \frac{}{\Gamma \vdash \mathbf{diverge} : \tau}$$

Fig. 25. Typing rules for general language constructs

$\mathbf{e} : \boldsymbol{\tau}$	**constraints**	$[\![\boldsymbol{e} : \boldsymbol{\tau}]\!]$
$\mathbf{diverge} : \tau$		$\lambda A.0$
$v : \tau$		$\delta_{v:\tau}$
$\mathbf{let}\ x = e_1 : \tau_1\ \mathbf{in}\ e_2 : \tau_2 : \tau_2$	$e_1 = v$ e_1 no val.	$\delta_{e_2[v/x]:\tau_2}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{\mathbf{let}\ x=g\ \mathbf{in}\ e_2:\tau_2:\tau_2}$
$\mathbf{if}\ e_1 : \mathbf{bool}\ \mathbf{then}\ e_2 : \tau\ \mathbf{else}\ e_3 : \tau : \tau$	$e_1 = \text{true}$ $e_1 = \text{false}$ o.w.	$\delta_{e_2:\tau}$ $\delta_{e_3:\tau}$ $[\![e_1 : \mathbf{bool}]\!] \gg\!= \lambda g.\delta_{\mathbf{if}\ g\ \mathbf{then}\ e_2:\tau\ \mathbf{else}\ e_3:\tau:\tau}$
$\mathbf{observe}\ (e : \mathbf{bool}) : \mathbf{unit}$	$e = \text{true}$ $e = \text{false}$ o.w.	$\delta_{\mathbf{unit}}$ $\delta_{\bot}$ $[\![e : \mathbf{bool}]\!] \gg\!= \lambda g.\delta_{\mathbf{observe}\ (g:\mathbf{bool}):\mathbf{unit}}$
$e_1 : \tau_1 < e_2 : \tau_2 : \mathbf{bool}$ for $\tau_1 = \mathbf{float}[B; V_1], \tau_2 = \mathbf{float}[B; V_2]$ or $\tau_1 = \tau_2 = \mathbf{fin}(n)$	$e_1 = v_1, e_2 = v_2, v_1 < v_2$ $e_1 = v_1, e_2 = v_2, v_1 \geq v_2$ $e_1 = v_1, e_2 \neq v_2$ o.w.	$\delta_{\text{true}:\mathbf{bool}}$ $\delta_{\text{false}:\mathbf{bool}}$ $[\![e_2 : \tau_2]\!] \gg\!= \lambda g.\delta_{v_1:\tau_1<g:\mathbf{bool}}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{g<e_2:\tau_2:\mathbf{bool}}$
$\mathbf{uniform}(e_1 : \tau_1, e_2 : \tau_2) : \mathbf{float}[B; V]$	$e_1 = v_1, e_2 = v_2, v_1 \leq v_2$ $e_1 = v_1, e_2 = v_2, v_1 > v_2$ $e_1 = v_1, e_2 \neq v_2$ o.w.	$\text{Uniform}(v_1, v_2) \gg\!= \lambda v.\delta_{(v:\mathbf{float}[B;V])}$ $[\![\mathbf{diverge} : \tau]\!]$ $[\![e_2 : \tau_2]\!] \gg\!= \lambda g.\delta_{\mathbf{uniform}(v_1:\tau_1,g):\tau}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{\mathbf{uniform}(g,e_2:\tau_2):\tau}$
$\mathbf{discrete}(v_0, \ldots, v_{n-1}) : \mathbf{fin}(n)$	~~$\sum_i v_i = 1$~~	$\text{Discrete}(v_0, \ldots, v_{n-1})$
$\mathtt{DC}_{B,B_1,B_2,V_1,V_2}(e_1 : \tau_1, e_2 : \tau_2) : \mathbf{fin}(\lvert B\rvert + 1)$ $\tau = \mathbf{fin}(\lvert B\rvert + 1)$, $\tau_1 = \mathbf{fin}(\lvert B_1\rvert + 1), \tau_2 = \mathbf{fin}(\lvert B_2\rvert + 1)$, $V_1 = \{v_1, \ldots, v_n\}, V_2 = \{w_1, \ldots, w_m\}$	$e_1 = \mathcal{D}[\![v_i : \mathbf{float}[B_1; V_1]]\!]$, $e_2 = \mathcal{D}[\![w_j : \mathbf{float}[B_2; V_2]]\!]$ $e_1 = v', e_2 \neq w'$ o.w.	$[\![\mathcal{D}[\![\mathbf{uniform}(v_i, w_j) : \mathbf{float}[B; \top]]\!] : \tau]\!]$ $[\![e_2 : \tau_2]\!] \gg\!= \lambda g.\delta_{\mathtt{DC}_{B,B_1,B_2,V_1,V_2}(v':\tau_1,g)}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{\mathtt{DC}_{B,B_1,B_2,V_1,V_2}(g,e_2:\tau_2)}$

Table 1. Small-step operational semantics. In the last case, the existence of suitable v_i and w_j is guaranteed since recovers(B_i, V_i) and $e_i \in V_i$ for $i \in \{1, 2\}$. Here, every function of the form $f(g) = \lambda g.\delta_{\ldots}$ is extended to $f_\bot$ in the sense of Definition F.29.

for every pairwise disjoint family $(A_i)_i$, and total mass $f(x)(Y) = 0 \leq 1$.

(2) Measurable function. For any $A \in \mathcal{F}_Y$, the map $x \mapsto f(x)(A) = 0$ is the constant-zero function on X, hence measurable.

Thus both conditions hold, so f is a Markov kernel. □

Lemma F.33 (Composition of Markov kernels). *Let (X, Σ_X), (Y, Σ_Y), and (Z, Σ_Z) be measurable spaces. If*

$$P : X \to \mathcal{D}(Y) \quad \textit{and} \quad Q : Y \to \mathcal{D}(Z)$$

are Markov kernels, then their composition

$$(P \gg\!= Q)(A) \ := \ \int_{x \in supp(P)} Q(x)(A)\, P(dx)$$

defines a Markov kernel

$$P \gg\!= Q : X \to \mathcal{D}(Z).$$

Proof. See [30, Lemma 1.38]. □

Equipped with these notions, we define the small-step semantics of our language:

Definition F.34. The small-step semantics $[\![e : \tau]\!] \in \mathcal{D}(Expr_\tau + \bot)$ is defined recursively on the structure of $e : \tau$ by the rules in Table 1 and Table 2.

In the Discrete row, the typing premises ensure that the displayed weights form a nonempty probability vector. Hence $\mathrm{Discrete}(p_0, \ldots, p_{n-1}) = \sum_{i=0}^{n-1} p_i \, \delta_{i_{\mathbb{R}^n}}$ is a probability measure in $\mathcal{D}(Expr_{\mathtt{fin}(n)} + \bot)$. The missing probability mass in $[\![e : \tau]\!]$ is the probability of encountering an error due to ill-formed parameters fed into distributions (e.g., $\mathtt{uniform}(0.8, 0.1)$). $[\![e : \tau]\!](\bot)$ is the probability of encountering an observation failure.

Lemma F.35 (Measurability of constant extension map). *Let $A \subseteq \mathbb{R}^n$ be equipped with the subspace σ-algebra*

$$\mathcal{F}_A := \{A \cap M \mid M \in \mathcal{B}(\mathbb{R}^n)\},$$

and let $B \subseteq \mathbb{R}^d$ be equipped with the subspace σ-algebra

$$\mathcal{F}_B := \{B \cap N \mid N \in \mathcal{B}(\mathbb{R}^d)\}.$$

Fix constant vectors $c_1 \in \mathbb{R}^{m_1}, \ldots, c_k \in \mathbb{R}^{m_k}$. Define

$$g : \mathbb{R}^n \to \mathbb{R}^d$$

to be any map obtained by concatenating blocks consisting of

- *copies of the input vector $\vec{r} \in \mathbb{R}^n$, and*
- *the constant vectors $c_1, \ldots, c_k$,*

in some fixed order. That is, $g(\vec{r})$ is formed by inserting the vectors $\vec{r}$ and c_j into a fixed sequence of blocks.

Assume $g(A) \subseteq B$ and define $\phi := g|_A : A \to B$. Then ϕ is measurable.

Example F.36 (Constant extension maps). The constant extension map allows constants to be inserted and the input vector $\vec{r} \in \mathbb{R}^n$ to appear multiple times. The following are examples of such maps:

- $g : \mathbb{R}^n \to \mathbb{R}^n, \quad g(\vec{r}) = \vec{r}$
- $g : \mathbb{R}^n \to \mathbb{R}^{n+m_1}, \quad g(\vec{r}) = (\vec{r}, c_1), \quad c_1 \in \mathbb{R}^{m_1}$
- $g : \mathbb{R}^n \to \mathbb{R}^{m_1+n+m_2}, \quad g(\vec{r}) = (c_1, \vec{r}, c_2), \quad c_1 \in \mathbb{R}^{m_1}, \ c_2 \in \mathbb{R}^{m_2}$
- $g : \mathbb{R}^n \to \mathbb{R}^{n+m_2}, \quad g(\vec{r}) = (\vec{r}, c_2), \quad c_2 \in \mathbb{R}^{m_2}$
- $g : \mathbb{R}^n \to \mathbb{R}^{m_1+n+m_2+n+m_3+n}, \quad g(\vec{r}) = (c_1, \vec{r}, c_2, \vec{r}, c_3, \vec{r}), \quad c_i \in \mathbb{R}^{m_i}$

Proof. We show that $\phi^{-1}(S) \in \mathcal{F}_A$ for each $S \in \mathcal{F}_B$.

Fix $S \in \mathcal{F}_B$. By definition of $\mathcal{F}_B$, there exists $N \in \mathcal{B}(\mathbb{R}^d)$ such that $S = B \cap N$. We compute:

$$\begin{aligned}
\phi^{-1}(S) &= \{\vec{r} \in A \mid \phi(\vec{r}) \in B \cap N\} \\
&= \{\vec{r} \in A \mid \phi(\vec{r}) \in B \cap \phi(\vec{r}) \in N\} && \text{(definition of } \cap) \\
&= \{\vec{r} \in A \mid \phi(\vec{r}) \in N\} && (\phi(\vec{r}) \in B \text{ for all } \vec{r} \in A) \\
&= \{\vec{r} \in A \mid g(\vec{r}) \in N\} && \text{(definition of } \phi) \\
&= \{\vec{r} \in A \mid g(\vec{r}) \in N\} \cap \mathbb{R}^n && (\vec{r} \text{ ranges over } A \subseteq \mathbb{R}^n) \\
&= A \cap \{\vec{r} \in \mathbb{R}^n \mid g(\vec{r}) \in N\}. && \text{(rewrite as } \cap \text{ with } A)
\end{aligned}$$

It remains to show that $\{\vec{r} \in \mathbb{R}^n \mid (\vec{r}, \vec{c}) \in N\} \in \mathcal{B}(\mathbb{R}^n)$. Define the total constant-extension map

$$g\colon \mathbb{R}^n \to \mathbb{R}^d.$$

We claim g is continuous. A map into a product space is continuous if and only if each coordinate function is continuous [36, Theorem 19.6]. The coordinate functions of g are: